The Contemporary Ledger: Applying the Framework to the Present

Macharia Barii

14 October 2026

Abstract

The classical typology of extraction—tribute, corvée, rent, slavery, liturgy—does not exhaust the modern ledger. This essay extends the framework of *The Essence of Civilization* to the present. The extension is structural rather than analogical: the same tuple (contributions, output, shares, exit cost, voice) classifies modern relations as well as ancient ones. We introduce thirteen contemporary relation types, identify three structural shifts that distinguish the present—exit asymmetry, the biosphere as ultimate host, and legitimacy decay without collapse—and state the formal extensions that follow, most importantly the exit-asymmetry index, the compliance-by-atomization equilibrium, and the failure of the conservation corollary under biospheric predation. Six non-obvious findings fall out. The framework does not predict which future obtains, but it specifies the parameters that decide, and names the discount rate as the single most consequential. The companion Rust pipeline is extended and the complete source reproduced in Appendix A.

Contents

1 New Relation Types **1**

2 The Structural Novelty of the Present **2**
2.1 Shift 1: Exit asymmetry 2
2.2 Shift 2: The scale of the host 2
2.3 Shift 3: Legitimacy decay without collapse 2

3 Formal Extensions **3**

4 Non-Obvious Findings **3**

5 Futures **5**

6 Rust Extension: Modern Relation Types **5**

A Complete Source **5**

B Sample Run **25**

1 New Relation Types

The classical typology does not exhaust the modern ledger. The framework extends naturally by adding relation types that share the same structural logic but differ in mechanism. Each is characterized by the same tuple: contributions, output, shares, exit cost, voice. Table 1 lists the extension.

Relation	Historical analogue	Mechanism	Classification
Platform labor	Wage labor + corvée	Algorithmic scheduling, no employment contract	Parasitic
Sovereign debt	Debt peonage	Conditionality, IMF programs, bond markets	Parasitic
Supply-chain extraction	Colonial tribute	Tiered subcontracting, price pressure	Parasitic
Data extraction	Caste labor	Free labor generating proprietary value	Parasitic
Carbon extraction	Conquest	Atmosphere as sink, future generations as host	Predatory
Tax-haven arbitrage	Feudal exemption	Jurisdictional exit by capital, not labor	Parasitic
Intellectual-property rent	Rentier landlordism	Legal monopoly on knowledge	Parasitic
Visa and border regimes	Serfdom	Exit cost imposed on labor, not capital	Parasitic
Care labor	Unpaid domestic labor	Gender-coded, invisible in GDP	Parasitic
Predatory finance	Debt peonage	High-interest credit to the poor	Parasitic/predatory
Conflict minerals	Slavery	Forced labor in extraction zones	Predatory
Fisheries depletion	Looting	Common-pool resource extraction	Predatory
Digital attention	Tithe	Time extracted in exchange for "free" service	Parasitic
Religious intermediation	Temple tithe	Signal monopoly over an unverifiable counterparty	Indeterminate/parasitic

Table 1: Contemporary relation types and their classical analogues.

2 The Structural Novelty of the Present

What distinguishes the contemporary ledger is not the existence of parasitic and predatory relations—those have always existed—but three structural shifts.

2.1 Shift 1: Exit asymmetry

Capital has near-zero exit costs: wire transfers, offshore accounts, flag-of-convenience registration. Labor has high exit costs: visas, language, family, borders. The modern ledger has inverted the classical relation between mobile elites and immobile producers. In feudalism, the lord was bound to the land as much as the serf. In the contemporary order, the elite is functionally stateless while the producer is legally trapped.

Formally, the exit ceiling is $\tau^{\text{exit}} = 1 - \sqrt{2(\bar{a}_A + k)}$. As $k_{\text{capital}} \to 0$ and $k_{\text{labor}} \to 1$, the extraction ceiling on capital approaches zero while the ceiling on labor approaches its maximum. The result is a structural transfer from labor to capital that requires no conspiracy, only the asymmetry of exit.

2.2 Shift 2: The scale of the host

The biosphere is now the ultimate host. Carbon extraction is a predatory relation in the strict sense: the host's capacity to reproduce is declining. Unlike previous predatory relations, however, the host cannot be replaced by conquest, and the predator cannot emigrate to a new planet. Predation is therefore self-terminating in a way that previous predatory orders were not.

2.3 Shift 3: Legitimacy decay without collapse

Previous civilizations faced a tight coupling: legitimacy fell, conflict rose, order collapsed. Contemporary orders have decoupled legitimacy from collapse through three technologies:

- **Surveillance:** repression R is cheaper than ever; R substitutes for legitimacy L.
- **Narrative fragmentation:** no shared public sphere means no shared grievance; injustice is atomized.
- **Welfare residue:** enough redistribution to prevent absolute immiseration without addressing relative deprivation.

The result is a parasitic equilibrium with low legitimacy and low conflict—a state the classical theory did not anticipate, because it assumed that legitimacy collapse would produce conflict. It produces, instead, *managed decay*.

3 Formal Extensions

Definition 3.1 (Exit-asymmetry index)**.** For an order, define

$$A = \frac{k_{\text{labor}}}{k_{\text{capital}}}.$$

When $A \gg 1$, capital is mobile and labor is captive. The extraction ceiling on capital is near zero; on labor, near its maximum.

Proposition 3.2 (Structural transfer)**.** Under exit asymmetry A, the equilibrium distribution of surplus shifts toward the mobile factor by an amount increasing in A and decreasing in the voice of the immobile factor.

Proposition 3.3 (Managed decay)**.** If surveillance technology raises R at lower cost than legitimacy L, and if narrative fragmentation raises the cost of collective voice X, then there exists a parasitic equilibrium with $L < L^*$ and $C \approx 0$. The order is stable but not legitimate.

Proposition 3.4 (Biospheric predation)**.** If the host is unique and non-substitutable, predatory extraction is not merely self-limiting but self-terminating. The optimal extraction rate under a finite horizon differs from the infinite-horizon case: impatience dominates, and the conservation corollary fails.

Proposition 3.5 (Legitimacy without belief)**.** In a surveillance order, legitimacy can be replaced by predictive compliance: actors comply not because they believe the order is just, but because they cannot coordinate resistance. This is a new equilibrium type, distinct from symbiotic, parasitic, and predatory. Call it *compliance by atomization*.

4 Non-Obvious Findings

Finding 4.1 (Stable but not legitimate). Conventional political science measures stability by the absence of conflict and legitimacy by survey data. Both can be high while the injustice gap G is large, because surveillance suppresses the translation of grievance into conflict. The framework predicts that a small shock—a legitimacy event, a communication technology that defeats surveillance, a fiscal crisis that ends welfare residue—can move the system from compliance by atomization to open conflict in a short interval. The stability is real but thin.

Finding 4.2 (Exit asymmetry is the central fact). Most debates about inequality focus on shares s_i. The framework says the deeper variable is k_i. Capital's near-zero exit cost is the structural feature that makes all other extraction possible. A just order requires not only redistribution of shares but equalization of exit costs—which is why capital controls, labor mobility, and universal basic services are structural justice mechanisms, not merely policy preferences.

Finding 4.3 (The biosphere is the first host that cannot be conquered). Every previous predatory relation had an outside: new lands, new peoples, new resources. The biosphere has no outside. The conservation corollary—patient predators conserve—therefore applies with maximum force to carbon extraction, and the discount rate r of the current order is the single most consequential parameter in the ledger. Every other variable is downstream of it.

Finding 4.4 (Legitimacy is being replaced by architecture). Traditional legitimacy was a belief (divine right, Mandate of Heaven, popular sovereignty). Contemporary legitimacy is increasingly an architecture (surveillance, algorithmic curation, platform design, credit scoring). This is not a moral decline; it is a substitution. As L falls and R rises, the cost of injustice does not rise as fast as classical theory predicts, because R substitutes for L. This is why injustice can accumulate without producing the classical response.

Finding 4.5 (The hidden history of the present). The hidden history of the present is the labor behind the digital ledger. Just as the pyramids conceal the corvée and the code of Hammurabi conceals the debt peons, the smartphone conceals the cobalt miner, the algorithm conceals the data annotator, and the supply chain conceals the garment worker. The achievement–labor gap applies directly: the achievement is visible, the labor is not. The framework does not need new methods to detect this; it needs the same method applied to a new ledger.

Finding 4.6 (Religion is a legitimacy technology, and its indeterminacy is its power). Religion is not an exception to the ledger; it is the ledger's cheapest legitimacy technology. Its counterparty D is unobservable—no capacity, reproduction threshold, exit cost, or voice can be measured, and the promised output is not verifiable—so classification is *indeterminate* on the divine side. The theory neither affirms nor denies D; it registers the earthly relation (worshipper $\rightarrow$ institution), which it classifies normally. Three consequences follow. *Solidification:* a religious relation feeds ideological input $I > 0$ into $\dot{L} = -\lambda G + \mu I$, raising L and lowering the cost of compliance $\Phi(G, L)$—it does not add force, it subtracts the price of force. *Reformation:* a rival legitimacy technology that lowers the incumbent's I drives $L < L^*$ and makes the old order infeasible with no change in the balance of force, so a new religion can replace an old order by reallocating the legitimacy stock rather than conquering it. *Deception:* because the output is unverifiable, the intermediary holds a *signal monopoly*—a principal–agent problem with hidden action and hidden information—and can invoke D to justify any transfer τ. The same indeterminacy that makes religion a cheap legitimacy technology is what makes it manipulable; this is why power reaches for it. The modern successor is legitimacy by *architecture* (Finding 4.4): where the intermediary once spoke for an unverifiable god, the platform now speaks for an unverifiable algorithm, and the signal monopoly is unchanged.

Religion is one legitimacy technology among several substitutes. Table 2 gives the contemporary instances of the full taxonomy, and the auditability ordering of the theory paper predicts where manipulation concentrates.

Table 2: Legitimacy technologies and their contemporary instances.

Type	**Contemporary instance**	**Acts on**
Belief / ideological	nationalism, market ideology, techno-optimism, brand and lifestyle politics	L (via I)
Procedural / voice	elections, courts, transparency and audit regimes, "stakeholder" consultation	G, D
Performance / output	growth, welfare residue, security, crisis competence	G
Coercion-substitute	surveillance, algorithmic curation, credit scoring, platform design	Φ (via R)
Temporal / institutional	corporate continuity, property rights, credible commitment, sovereign ratings	extraction, G
Ceremonial / spectacular	celebrity culture, mega-events, sport, mass culture	L

5 Futures

The dynamical system admits several trajectories from the current state. Which one obtains depends on a small number of parameters (Table 3).

Trajectory	**Condition**	**Signature**
Managed decay	R rises faster than G; X remains low	Stability with declining L and rising G
Reform	X rises; justice mechanism J activates	Redistribution, exit equalization
Fragmentation	G exceeds threshold; L collapses in some regions	Conflicting regional orders
Biospheric collapse	r remains high; biospheric $K \to 0$	Cascading failure across all relations
Symbiotic transition	r falls; exit asymmetry falls; J operates	New equilibrium

Table 3: Trajectories of the contemporary ledger and their conditions.

The framework does not predict which will occur. It specifies the parameters that determine the outcome, and it says that the most consequential parameter—the one with the largest effect on all others—is the discount rate r. A civilization that lowers its r is one that lowers its extraction rate, conserves its hosts, and rebalances its ledger. A civilization that raises its r consumes its hosts and hastens its own collapse.

6 Rust Extension: Modern Relation Types

The companion analyzer is extended to the contemporary ledger. The extension adds the modern relation types to the unified classifier, an exit-asymmetry index, a compliance-by-atomization detector, and a biospheric-predation check. The contemporary worked example exercises all three. The complete source is in Appendix A and a sample run—which reports an exit-asymmetry index of 42.5 and detects both compliance by atomization and biospheric predation—is in Appendix B.

A Complete Source

```rust
//! civilization_analyzer.rs
//!
//! Algorithmic analysis of civilizations as ledgers of reciprocal relations.
//!
//! This program implements, in full functional form, the framework developed in
//! "The Essence of Civilization: A Formal Theory of Reciprocal Relations,
//! Justice, and Conflict" together with its two extensions:
//!
//!   * "Algorithmic Analysis of Civilizations" (the classical pipeline), and
//!   * "The Contemporary Ledger" (the modern relation types and diagnostics).
//!
//! Pipeline:
//!     ingest -> classify -> assess -> detect hidden history
//!            -> project dynamics -> report -> contemporary diagnostics
//!
//! Build:  cargo build --release
//! Run:    cargo run --release
//!
//! The classical and contemporary relation types share a single enum so that a
//! single ledger can mix tribute and platform labour, corvee and carbon
//! extraction. This is what makes the contemporary diagnostics functional
//! rather than illustrative.

use std::collections::HashMap;
use std::fmt::Write as _;

```

```rust
// ============================================================================
// 1. TYPES
// ============================================================================

pub type ActorId = usize;
pub type RelationId = usize;

/// Every relation in the ledger, classical and contemporary.
///
/// The structural logic is identical across the two families: each is a tuple
/// (contributions, output, shares, exit cost, voice). Only the mechanism
/// differs. Keeping them in one enum lets the classifier, the hidden-history
/// detector, and the dynamical projection treat them uniformly.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum RelationKind {
    // -- classical ------------------------------------------------
    Exchange,
    MutualAid,
    Tribute,
    Taxation,
    Corvee,
    Rent,
    DebtPeonage,
    Slavery,
    CasteLabor,
    PatronClient,
    WageLabor,
    FeudalService,
    Liturgy,
    ColonialTribute,
    Conquest,
    Occupation,
    Genocide,
    // -- contemporary ---------------------------------------------
    PlatformLabor,
    SovereignDebt,
    SupplyChainExtraction,
    DataExtraction,
    CarbonExtraction,
    TaxHavenArbitrage,
    IntellectualPropertyRent,
    VisaRegime,
    CareLabor,
    PredatoryFinance,
    ConflictMinerals,
    FisheriesDepletion,
    DigitalAttention,
}

impl RelationKind {
    /// A coercive relation is one that a party cannot freely refuse on terms of
    /// its own choosing.
    pub fn coercive(&self) -> bool {
        use RelationKind::*;
        matches!(
            self,
            // classical
            Tribute | Taxation | Corvee | Rent | DebtPeonage
                | Slavery | CasteLabor | FeudalService
                | ColonialTribute | Conquest | Occupation
            // contemporary
                | PlatformLabor | SovereignDebt | SupplyChainExtraction
                | DataExtraction | CarbonExtraction | TaxHavenArbitrage
                | IntellectualPropertyRent | VisaRegime | CareLabor
                | PredatoryFinance | ConflictMinerals | FisheriesDepletion
                | DigitalAttention
        )
    }

    /// A labour-extractive relation is one whose costs fall on human capacity
    /// to reproduce.
    pub fn labor_extractive(&self) -> bool {
        use RelationKind::*;
```

```
        matches!(
            self,
            Corvee | Slavery | DebtPeonage | CasteLabor
                | FeudalService | ColonialTribute
                | PlatformLabor | CareLabor | ConflictMinerals
        )
    }

    /// Relations that consume the host's capacity to reproduce (Definition 4.3).
    pub fn predatory(&self) -> bool {
        use RelationKind::*;
        matches!(
            self,
            Slavery | Conquest | Genocide
                | CarbonExtraction | ConflictMinerals | FisheriesDepletion
        )
    }

    /// The biosphere cannot be conquered or replaced: relations whose host is
    /// the biosphere are the ones for which the conservation corollary fails.
    pub fn is_biospheric(&self) -> bool {
        use RelationKind::*;
        matches!(self, CarbonExtraction | FisheriesDepletion)
    }

    pub fn label(&self) -> &'static str {
        use RelationKind::*;
        match self {
            Exchange => "exchange",
            MutualAid => "mutual aid",
            Tribute => "tribute",
            Taxation => "taxation",
            Corvee => "corvee",
            Rent => "rent",
            DebtPeonage => "debt peonage",
            Slavery => "slavery",
            CasteLabor => "caste labor",
            PatronClient => "patron-client",
            WageLabor => "wage labor",
            FeudalService => "feudal service",
            Liturgy => "liturgy",
            ColonialTribute => "colonial tribute",
            Conquest => "conquest",
            Occupation => "occupation",
            Genocide => "genocide",
            PlatformLabor => "platform labor",
            SovereignDebt => "sovereign debt",
            SupplyChainExtraction => "supply-chain extraction",
            DataExtraction => "data extraction",
            CarbonExtraction => "carbon extraction",
            TaxHavenArbitrage => "tax-haven arbitrage",
            IntellectualPropertyRent => "IP rent",
            VisaRegime => "visa regime",
            CareLabor => "care labor",
            PredatoryFinance => "predatory finance",
            ConflictMinerals => "conflict minerals",
            FisheriesDepletion => "fisheries depletion",
            DigitalAttention => "digital attention",
        }
    }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum FlowStatus {
    Symbiotic,
    Parasitic,
    Predatory,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum RelationClass {
    Symbiotic,
    Parasitic,
```

```
    Predatory,
}

// ---------------------------------------------------------------------
// Actor
// ---------------------------------------------------------------------

#[derive(Debug, Clone)]
pub struct Actor {
    pub id: ActorId,
    pub name: String,
    pub role: String,

    /// Participation threshold: what the actor could obtain outside.
    pub participation_threshold: f64,
    /// Reproduction threshold: minimum to sustain capacity.
    pub reproduction_threshold: f64,

    /// Current capacity K_i and ceiling.
    pub capacity: f64,
    pub max_capacity: f64,

    /// Legitimacy L_i, conflict C_i, repression R_i, ideology I_i.
    pub legitimacy: f64,
    pub conflict: f64,
    pub repression: f64,
    pub ideology: f64,

    /// Exit cost k_i and voice v_i (aggregate across relations).
    pub exit_cost: f64,
    pub voice: f64,

    /// Growth parameters.
    pub growth_rate: f64,
    pub depreciation: f64,
    pub conflict_damage: f64,
}

impl Actor {
    pub fn new(id: ActorId, name: &str, role: &str) -> Self {
        Actor {
            id,
            name: name.to_string(),
            role: role.to_string(),
            participation_threshold: 0.05,
            reproduction_threshold: 0.0,
            capacity: 1.0,
            max_capacity: 2.0,
            legitimacy: 0.6,
            conflict: 0.0,
            repression: 0.0,
            ideology: 0.3,
            exit_cost: 0.5,
            voice: 0.2,
            growth_rate: 0.05,
            depreciation: 0.02,
            conflict_damage: 0.10,
        }
    }
}

// ---------------------------------------------------------------------
// Relation
// ---------------------------------------------------------------------

#[derive(Debug, Clone)]
pub struct Relation {
    pub id: RelationId,
    pub from: ActorId,
    pub to: ActorId,
    pub kind: RelationKind,

    pub output: f64,
```

```
    pub contribution_from: f64,
    pub contribution_to: f64,
    pub share_from: f64,
    pub share_to: f64,

    pub exit_cost_from: f64,
    pub exit_cost_to: f64,
    pub voice_from: f64,
    pub voice_to: f64,

    pub description: String,
}

impl Relation {
    #[allow(clippy::too_many_arguments)]
    pub fn new(
        id: RelationId,
        from: ActorId,
        to: ActorId,
        kind: RelationKind,
        output: f64,
        share_from: f64,
        share_to: f64,
        contribution_from: f64,
        contribution_to: f64,
    ) -> Self {
        Relation {
            id,
            from,
            to,
            kind,
            output,
            contribution_from,
            contribution_to,
            share_from,
            share_to,
            exit_cost_from: 0.0,
            exit_cost_to: 0.5,
            voice_from: 0.5,
            voice_to: 0.1,
            description: String::new(),
        }
    }

    pub fn net_gain_from(&self) -> f64 {
        self.share_from * self.output - self.contribution_from
    }

    pub fn net_gain_to(&self) -> f64 {
        self.share_to * self.output - self.contribution_to
    }
}

// ---------------------------------------------------------------------
// Achievement (a product of the ledger)
// ---------------------------------------------------------------------

#[derive(Debug, Clone)]
pub struct Achievement {
    pub name: String,
    pub domain: String,

    /// Estimated labor-years embodied in the achievement.
    pub labor_years: f64,
    /// Estimated surplus consumed (arbitrary units).
    pub surplus_cost: f64,

    /// Actors explicitly named in the official record as builders.
    pub credited: Vec<ActorId>,
    /// Actors explicitly named as laborers.
    pub laborers_named: Vec<ActorId>,
}

```

```rust
// ---------------------------------------------------------------------
// Civilization
// ---------------------------------------------------------------------

#[derive(Debug, Clone, Default)]
pub struct Civilization {
    pub name: String,
    pub period: String,

    pub actors: HashMap<ActorId, Actor>,
    pub relations: Vec<Relation>,
    pub achievements: Vec<Achievement>,
}

impl Civilization {
    pub fn new(name: &str, period: &str) -> Self {
        Civilization {
            name: name.to_string(),
            period: period.to_string(),
            ..Default::default()
        }
    }

    pub fn add_actor(&mut self, actor: Actor) {
        self.actors.insert(actor.id, actor);
    }

    pub fn add_relation(&mut self, relation: Relation) {
        self.relations.push(relation);
    }

    pub fn add_achievement(&mut self, a: Achievement) {
        self.achievements.push(a);
    }

    pub fn total_surplus(&self) -> f64 {
        self.relations
            .iter()
            .map(|r| r.output - r.contribution_from - r.contribution_to)
            .sum()
    }

    pub fn net_gain_of(&self, actor_id: ActorId) -> f64 {
        let mut n = 0.0;
        for r in &self.relations {
            if r.from == actor_id {
                n += r.net_gain_from();
            }
            if r.to == actor_id {
                n += r.net_gain_to();
            }
        }
        n
    }

    pub fn total_coercive_labor_years_per_period(&self) -> f64 {
        self.relations
            .iter()
            .filter(|r| r.kind.labor_extractive())
            .map(|r| r.contribution_to)
            .sum()
    }

    /// Actors whose role marks them as controllers of mobile capital.
    pub fn capital_exit_cost(&self) -> f64 {
        self.actors
            .values()
            .filter(|a| {
                a.role.contains("elite")
                    || a.role.contains("ruler")
                    || a.role.contains("capital")
                    || a.role.contains("firm")
            })
```

```
            .map(|a| a.exit_cost)
            .fold(f64::INFINITY, f64::min)
            .max(0.01)
    }

    /// Actors whose role marks them as immobile labour.
    pub fn labor_exit_cost(&self) -> f64 {
        self.actors
            .values()
            .filter(|a| {
                a.role.contains("producer")
                    || a.role.contains("worker")
                    || a.role.contains("unfree")
                    || a.role.contains("carer")
                    || a.role.contains("data subject")
            })
            .map(|a| a.exit_cost)
            .fold(0.0_f64, f64::max)
            .max(0.01)
    }
}

// ============================================================================
// 2. CLASSIFICATION
// ============================================================================

fn flow_status(net_gain: f64, actor: &Actor) -> FlowStatus {
    if net_gain < actor.reproduction_threshold {
        FlowStatus::Predatory
    } else if net_gain < actor.participation_threshold {
        FlowStatus::Parasitic
    } else {
        FlowStatus::Symbiotic
    }
}

fn classify_relation(r: &Relation, actors: &HashMap<ActorId, Actor>) -> RelationClass {
    let a_from = actors
        .get(&r.from)
        .expect("relation refers to unknown actor (from)");
    let a_to = actors
        .get(&r.to)
        .expect("relation refers to unknown actor (to)");

    let s_from = flow_status(r.net_gain_from(), a_from);
    let s_to = flow_status(r.net_gain_to(), a_to);

    // The relation takes the worst status of its parties.
    if s_from == FlowStatus::Predatory || s_to == FlowStatus::Predatory {
        RelationClass::Predatory
    } else if s_from == FlowStatus::Parasitic || s_to == FlowStatus::Parasitic {
        RelationClass::Parasitic
    } else {
        RelationClass::Symbiotic
    }
}

// ============================================================================
// 3. AGGREGATE STATUS
// ============================================================================

#[derive(Debug, Clone)]
pub struct RelationAssessment {
    pub id: RelationId,
    pub kind: RelationKind,
    pub from: String,
    pub to: String,
    pub class: RelationClass,
    pub net_from: f64,
    pub net_to: f64,
    pub exploited_party: Option<String>,
    pub exploited_flow: Option<FlowStatus>,
}
```

```rust

#[derive(Debug, Clone)]
pub struct StatusReport {
    pub civilization: String,
    pub relations: Vec<RelationAssessment>,
    pub symbiotic_count: usize,
    pub parasitic_count: usize,
    pub predatory_count: usize,
    pub aggregate: RelationClass,
    pub aggregate_justification: String,
}

pub fn assess_status(c: &Civilization) -> StatusReport {
    let mut assessments = Vec::new();
    let mut sym = 0;
    let mut par = 0;
    let mut pred = 0;

    for r in &c.relations {
        let class = classify_relation(r, &c.actors);

        let a_from = &c.actors[&r.from];
        let a_to = &c.actors[&r.to];
        let n_from = r.net_gain_from();
        let n_to = r.net_gain_to();

        let (exploited_party, exploited_flow) = match class {
            RelationClass::Symbiotic => (None, None),
            _ => {
                let s_from = flow_status(n_from, a_from);
                let s_to = flow_status(n_to, a_to);
                // The exploited party is the one with the worse flow status.
                let from_rank = match s_from {
                    FlowStatus::Symbiotic => 2,
                    FlowStatus::Parasitic => 1,
                    FlowStatus::Predatory => 0,
                };
                let to_rank = match s_to {
                    FlowStatus::Symbiotic => 2,
                    FlowStatus::Parasitic => 1,
                    FlowStatus::Predatory => 0,
                };
                if from_rank < to_rank {
                    (Some(a_from.name.clone()), Some(s_from))
                } else {
                    (Some(a_to.name.clone()), Some(s_to))
                }
            }
        };

        match class {
            RelationClass::Symbiotic => sym += 1,
            RelationClass::Parasitic => par += 1,
            RelationClass::Predatory => pred += 1,
        }

        assessments.push(RelationAssessment {
            id: r.id,
            kind: r.kind,
            from: a_from.name.clone(),
            to: a_to.name.clone(),
            class,
            net_from: n_from,
            net_to: n_to,
            exploited_party,
            exploited_flow,
        });
    }

    // Aggregate: the worst category present, for analytical clarity, with
    // counts reported alongside.
    let aggregate = if pred > 0 {
        RelationClass::Predatory
```

```
    } else if par > 0 {
        RelationClass::Parasitic
    } else {
        RelationClass::Symbiotic
    };

    let aggregate_justification = format!(
        "{} symbiotic, {} parasitic, {} predatory relations. \
         Aggregate classification: {:?} (worst category present).",
        sym, par, pred, aggregate
    );

    StatusReport {
        civilization: c.name.clone(),
        relations: assessments,
        symbiotic_count: sym,
        parasitic_count: par,
        predatory_count: pred,
        aggregate,
        aggregate_justification,
    }
}

// ============================================================================
// 4. VIABILITY AND EXTRACTION BOUNDS
// ============================================================================

#[derive(Debug, Clone)]
pub struct ExtractionBounds {
    pub actor: String,
    pub monopolist_rate: f64,
    pub exit_ceiling: f64,
    pub effective_rate: f64,
    pub binding_constraint: &'static str,
}

pub fn extraction_bounds(actor: &Actor, external_exit_cost: f64) -> ExtractionBounds {
    let monopolist: f64 = 0.5;
    let k = external_exit_cost + actor.exit_cost;
    let exit_ceiling = 1.0 - (2.0 * (actor.participation_threshold + k)).sqrt().max(0.0);
    let exit_ceiling = exit_ceiling.clamp(0.0, 1.0);
    let effective = monopolist.min(exit_ceiling);
    let binding = if exit_ceiling < monopolist {
        "exit"
    } else {
        "revenue maximization"
    };
    ExtractionBounds {
        actor: actor.name.clone(),
        monopolist_rate: monopolist,
        exit_ceiling,
        effective_rate: effective,
        binding_constraint: binding,
    }
}

// ============================================================================
// 5. HIDDEN HISTORY DETECTION
// ============================================================================

#[derive(Debug, Clone)]
pub struct HiddenCandidate {
    pub headline: String,
    pub reasoning: String,
    pub confidence: f64,
    pub implicated_actors: Vec<String>,
    pub evidence: Vec<String>,
}

/// Detects what the official record is likely to have left out.
///
/// Heuristics:
/// 1. Achievement-labor gap: monuments imply labor, but the narrative
```

```rust
///    names elites, not laborers.
/// 2. Extraction gap: coercive relations are the likely hidden channel.
/// 3. Legitimacy shield: high ideological investment with high inequality
///    suggests suppression of grievance.
/// 4. Silent exit: high exit costs with low voice suggest a captive
///    population whose dissatisfaction is invisible.
pub fn detect_hidden_history(c: &Civilization) -> Vec<HiddenCandidate> {
    let mut out = Vec::new();

    // -- Heuristic 1: achievement-labor gap -----------------------------
    let total_achievement_labor: f64 = c.achievements.iter().map(|a| a.labor_years).sum();
    let named_laborers: usize = c
        .achievements
        .iter()
        .map(|a| a.laborers_named.len())
        .sum();
    let coercive_labor_per_period = c.total_coercive_labor_years_per_period();

    if total_achievement_labor > 0.0 && coercive_labor_per_period > 0.0 {
        // If achievements require more labor than the visible coercive
        // channels can plausibly supply, there is a hidden channel.
        let plausible_visible = coercive_labor_per_period * 10.0;
        if total_achievement_labor > plausible_visible {
            let mut evidence = vec![format!(
                "Achievements require ~{:.0} labor-years; coercive channels can supply ~{:.0}.",
                total_achievement_labor, plausible_visible
            )];
            if named_laborers == 0 {
                evidence.push(
                    "No laborers are named in the official record of achievements.".to_string(),
                );
            }
            out.push(HiddenCandidate {
                headline: "Unrecorded labor mobilization".to_string(),
                reasoning: "Achievements imply far more labor than the visible \
                            coercive channels account for. The gap suggests \
                            unrecorded corvee, slavery, or tribute."
                    .to_string(),
                confidence: 0.7,
                implicated_actors: c
                    .actors
                    .values()
                    .filter(|a| a.role.contains("producer") || a.role.contains("unfree"))
                    .map(|a| a.name.clone())
                    .collect(),
                evidence,
            });
        }
    }

    // -- Heuristic 2: extraction gap ------------------------------------
    // If some actors have high net gains and others are below their
    // participation thresholds, coercion is the likely hidden channel.
    for r in &c.relations {
        if r.kind.coercive() {
            let a_to = &c.actors[&r.to];
            let n_to = r.net_gain_to();
            if n_to < a_to.participation_threshold {
                let gap = a_to.participation_threshold - n_to;
                out.push(HiddenCandidate {
                    headline: format!("Suppressed grievance under {}", r.kind.label()),
                    reasoning: format!(
                        "{} receives {:.3} below its participation threshold \
                         in a coercive relation. Grievance of this magnitude \
                         is typically suppressed in the official record.",
                        a_to.name, gap
                    ),
                    confidence: (0.5 + gap).min(0.95),
                    implicated_actors: vec![a_to.name.clone()],
                    evidence: vec![format!(
                        "Net gain {:.3} vs. threshold {:.3}",
                        n_to, a_to.participation_threshold
                    )],
```

```rust
                });
            }
        }
    }

    // -- Heuristic 3: legitimacy shield ----------------------------------
    let avg_ideology: f64 =
        c.actors.values().map(|a| a.ideology).sum::<f64>() / c.actors.len().max(1) as f64;
    let avg_legitimacy: f64 =
        c.actors.values().map(|a| a.legitimacy).sum::<f64>() / c.actors.len().max(1) as f64;
    let max_share: f64 = c
        .relations
        .iter()
        .map(|r| r.share_from.max(r.share_to))
        .fold(0.0_f64, f64::max);

    if avg_ideology > 0.5 && avg_legitimacy > 0.5 && max_share > 0.75 {
        out.push(HiddenCandidate {
            headline: "Ideological pacification of inequality".to_string(),
            reasoning: "High ideological investment and legitimacy coexist with \
                        extreme concentration of shares. The legitimacy is likely \
                        a manufactured shield for extraction."
                .to_string(),
            confidence: 0.6,
            implicated_actors: vec![],
            evidence: vec![
                format!("Average ideology {:.2}", avg_ideology),
                format!("Average legitimacy {:.2}", avg_legitimacy),
                format!("Maximum share {:.2}", max_share),
            ],
        });
    }

    // -- Heuristic 4: silent exit ----------------------------------------
    let high_exit_low_voice: Vec<&Actor> = c
        .actors
        .values()
        .filter(|a| a.exit_cost > 0.6 && a.voice < 0.2)
        .collect();
    if !high_exit_low_voice.is_empty() {
        out.push(HiddenCandidate {
            headline: "Captive population with no record of dissent".to_string(),
            reasoning: "High exit costs and low voice imply that dissatisfaction \
                        cannot be expressed or observed. The absence of recorded \
                        grievance is evidence of suppression, not contentment."
                .to_string(),
            confidence: 0.65,
            implicated_actors: high_exit_low_voice
                .iter()
                .map(|a| a.name.clone())
                .collect(),
            evidence: high_exit_low_voice
                .iter()
                .map(|a| format!("{}: exit cost {:.2}, voice {:.2}", a.name, a.exit_cost, a.voice))
                .collect(),
        });
    }

    out
}

// ============================================================================
// 6. DYNAMICAL PROJECTION
// ============================================================================

#[derive(Debug, Clone)]
pub struct Params {
    pub alpha: f64,        // fairness weight on legitimacy
    pub fairness_star: f64,
    pub beta: f64,         // repression weight on legitimacy
    pub gamma: f64,        // ideology weight on legitimacy
    pub lambda: f64,       // legitimacy damping on conflict
    pub mu: f64,           // voice/exit damping on conflict
```

```rust
    pub kappa: f64,        // share adjustment rate
    pub dt: f64,
}

impl Default for Params {
    fn default() -> Self {
        Params {
            alpha: 1.0,
            fairness_star: 0.0,
            beta: 0.5,
            gamma: 0.8,
            lambda: 1.0,
            mu: 0.5,
            kappa: 0.05,
            dt: 0.1,
        }
    }
}

#[derive(Debug, Clone)]
pub struct TrajectoryPoint {
    pub t: f64,
    pub total_capacity: f64,
    pub avg_legitimacy: f64,
    pub avg_conflict: f64,
    pub injustice_gap: f64,
}

#[derive(Debug, Clone)]
pub struct TrajectoryReport {
    pub points: Vec<TrajectoryPoint>,
    pub final_state: TrajectoryPoint,
    pub phase: &'static str,
    pub threshold_distance: f64,
    pub counterintuitive_finding: Option<String>,
}

fn sigmoid(x: f64) -> f64 {
    1.0 / (1.0 + (-x).exp())
}

fn compute_injustice_gap(c: &Civilization) -> f64 {
    let mut g = 0.0;
    for (id, a) in &c.actors {
        let n = c.net_gain_of(*id);
        let gap = (a.participation_threshold - n).max(0.0);
        g += gap;
    }
    g
}

fn step(state: &mut [f64], c: &Civilization, params: &Params) {
    // State layout: [K_0..K_{n-1}, L_0..L_{n-1}, C_0..C_{n-1}]
    let n = c.actors.len();
    let ids: Vec<ActorId> = {
        let mut v: Vec<ActorId> = c.actors.keys().copied().collect();
        v.sort();
        v
    };

    // Compute net gains for each actor given current shares.
    let mut net_gains: HashMap<ActorId, f64> = HashMap::new();
    for id in &ids {
        net_gains.insert(*id, c.net_gain_of(*id));
    }

    // Fairness per actor: ratio of realized share to average share.
    let avg_share = 1.0 / n.max(1) as f64;
    let max_share = c
        .relations
        .iter()
        .map(|r| r.share_from.max(r.share_to))
        .fold(0.0_f64, f64::max);
```

```rust

    let mut derivatives = vec![0.0; 3 * n];

    for (i, id) in ids.iter().enumerate() {
        let a = &c.actors[id];

        // --- dK/dt ---
        let k = state[i];
        let logistic = a.growth_rate * k * (1.0 - k / a.max_capacity.max(1e-6));
        let net = net_gains.get(id).copied().unwrap_or(0.0);
        let depreciation = a.depreciation * k;
        let c_i = state[2 * n + i];
        let damage = a.conflict_damage * c_i;
        derivatives[i] = logistic + net - depreciation - damage;

        // --- dL/dt ---
        let fairness = if max_share > 0.0 {
            (avg_share / max_share - 1.0).clamp(-1.0, 1.0)
        } else {
            0.0
        };
        let l = state[n + i];
        derivatives[n + i] = params.alpha * (fairness - params.fairness_star)
            - params.beta * a.repression
            + params.gamma * a.ideology
            - 0.01 * l; // slow decay

        // --- dC/dt ---
        let deprivation = (max_share - avg_share).max(0.0);
        let power_asym = max_share;
        let voice_exit = a.voice + 1.0 / (1.0 + a.exit_cost);
        let z = deprivation + power_asym - params.lambda * l - params.mu * voice_exit;
        derivatives[2 * n + i] = sigmoid(z) - 0.05 * c_i;
    }

    let dt = params.dt;
    for i in 0..3 * n {
        state[i] = (state[i] + dt * derivatives[i]).max(0.0);
    }
}

pub fn project(c: &Civilization, params: &Params, t_end: f64) -> TrajectoryReport {
    let n = c.actors.len();
    let ids: Vec<ActorId> = {
        let mut v: Vec<ActorId> = c.actors.keys().copied().collect();
        v.sort();
        v
    };

    let mut state = vec![0.0; 3 * n];
    for (i, id) in ids.iter().enumerate() {
        let a = &c.actors[id];
        state[i] = a.capacity;
        state[n + i] = a.legitimacy;
        state[2 * n + i] = a.conflict;
    }

    let mut points = Vec::new();
    let mut t = 0.0;
    let mut steps = 0;

    while t <= t_end {
        let total_capacity: f64 = state[..n].iter().sum();
        let avg_legitimacy: f64 = state[n..2 * n].iter().sum::<f64>() / n.max(1) as f64;
        let avg_conflict: f64 = state[2 * n..].iter().sum::<f64>() / n.max(1) as f64;
        let g = compute_injustice_gap(c);

        points.push(TrajectoryPoint {
            t,
            total_capacity,
            avg_legitimacy,
            avg_conflict,
            injustice_gap: g,
```

```
        });

        step(&mut state, c, params);
        t += params.dt;
        steps += 1;
        if steps > 10_000 {
            break;
        }
    }

    let final_state = points
        .last()
        .cloned()
        .unwrap_or(TrajectoryPoint {
            t: 0.0,
            total_capacity: 0.0,
            avg_legitimacy: 0.0,
            avg_conflict: 0.0,
            injustice_gap: 0.0,
        });

    // Classify phase.
    let initial_capacity: f64 = c.actors.values().map(|a| a.capacity).sum();
    let phase = if final_state.total_capacity < 0.25 * initial_capacity {
        "collapse"
    } else if final_state.avg_conflict > 0.6 {
        "open conflict"
    } else if final_state.avg_legitimacy < 0.2 {
        "legitimacy crisis"
    } else if final_state.total_capacity > 1.1 * initial_capacity
        && final_state.avg_legitimacy > 0.4
    {
        "symbiotic expansion"
    } else if final_state.injustice_gap > 0.2 {
        "parasitic equilibrium"
    } else {
        "stable"
    };

    // Distance to nearest threshold: ratio of current to critical.
    let threshold_distance = (final_state.injustice_gap - 0.2).abs();

    // Counterintuitive finding: high legitimacy masking high injustice.
    let counterintuitive = if final_state.avg_legitimacy > 0.5 && final_state.injustice_gap > 0.25 {
        Some(
            "Legitimacy is high while the injustice gap is also high. \
             The order appears stable but is accumulating hidden grievance. \
             A small shock to legitimacy would trigger rapid conflict."
                .to_string(),
        )
    } else if final_state.avg_conflict < 0.05
        && final_state.total_capacity < 0.5 * initial_capacity
    {
        Some(
            "Conflict is low while capacity collapses. This is not peace; \
             it is exhaustion. The population lacks the surplus to resist."
                .to_string(),
        )
    } else {
        None
    };

    TrajectoryReport {
        points,
        final_state,
        phase,
        threshold_distance,
        counterintuitive_finding: counterintuitive,
    }
}

// ============================================================================
// 7. HEURISTIC TEXT INGESTION
```

```rust
// ========================================================================

/// Produces a skeleton Civilization from a text description.
///
/// This is deliberately heuristic: it identifies actor roles, coercive
/// relations, and named achievements, and constructs a plausible ledger.
/// The analyst is expected to refine the result before running the full
/// pipeline.
pub fn heuristic_parse(text: &str, name: &str, period: &str) -> Civilization {
    let mut c = Civilization::new(name, period);
    let lower = text.to_lowercase();

    // Actor keyword -> (role, share, participation_threshold, exit_cost,
    //                   voice, legitimacy, repression).
    let actor_keywords: &[(&str, &str, f64, f64, f64, f64, f64, f64)] = &[
        // -------- classical actors --------
        ("pharaoh", "ruler", 0.80, 0.05, 0.00, 0.90, 0.90, 0.30),
        ("king", "ruler", 0.75, 0.05, 0.00, 0.80, 0.85, 0.30),
        ("emperor", "ruler", 0.75, 0.05, 0.00, 0.80, 0.85, 0.30),
        ("priest", "clergy", 0.15, 0.05, 0.20, 0.50, 0.80, 0.10),
        ("noble", "elite", 0.30, 0.05, 0.20, 0.60, 0.70, 0.20),
        ("lord", "elite", 0.30, 0.05, 0.20, 0.60, 0.70, 0.20),
        ("bureaucrat", "official", 0.20, 0.05, 0.30, 0.50, 0.60, 0.20),
        ("merchant", "trader", 0.15, 0.05, 0.40, 0.40, 0.50, 0.10),
        ("artisan", "producer", 0.08, 0.05, 0.50, 0.25, 0.40, 0.20),
        ("peasant", "producer", 0.03, 0.05, 0.70, 0.10, 0.35, 0.20),
        ("farmer", "producer", 0.03, 0.05, 0.70, 0.10, 0.35, 0.20),
        ("serf", "producer", 0.02, 0.05, 0.90, 0.05, 0.30, 0.30),
        ("slave", "unfree", 0.00, 0.02, 1.00, 0.00, 0.20, 0.90),
        ("citizen", "member", 0.10, 0.05, 0.30, 0.70, 0.60, 0.20),
        ("soldier", "military", 0.08, 0.05, 0.40, 0.30, 0.50, 0.40),
        ("foreigner", "outsider", 0.04, 0.05, 0.80, 0.10, 0.30, 0.40),
        // -------- contemporary actors --------
        ("capital", "capital", 0.35, 0.05, 0.02, 0.85, 0.30, 0.30),
        ("bondholder", "capital", 0.30, 0.05, 0.02, 0.85, 0.30, 0.30),
        ("multinational", "firm", 0.30, 0.05, 0.05, 0.80, 0.35, 0.40),
        ("firm", "firm", 0.30, 0.05, 0.05, 0.80, 0.35, 0.40),
        ("platform", "platform", 0.30, 0.05, 0.15, 0.60, 0.40, 0.70),
        ("government", "state", 0.25, 0.05, 0.05, 0.70, 0.40, 0.80),
        ("state", "state", 0.25, 0.05, 0.05, 0.70, 0.40, 0.80),
        ("worker", "producer", 0.04, 0.05, 0.70, 0.15, 0.30, 0.70),
        ("gig", "producer", 0.03, 0.05, 0.75, 0.10, 0.30, 0.70),
        ("user", "data subject", 0.02, 0.05, 0.85, 0.05, 0.30, 0.70),
        ("data", "data subject", 0.02, 0.05, 0.85, 0.05, 0.30, 0.70),
        ("household", "household", 0.05, 0.05, 0.60, 0.20, 0.40, 0.50),
        ("carer", "carer", 0.02, 0.05, 0.80, 0.05, 0.30, 0.70),
        ("biosphere", "biosphere", 0.00, 0.00, 1.00, 0.00, 0.20, 0.20),
        ("periphery", "periphery", 0.03, 0.05, 0.80, 0.10, 0.30, 0.60),
    ];

    let mut id = 0;
    let mut seen_role: HashMap<&str, ActorId> = HashMap::new();
    for (kw, role, share, pthr, exit, voice, legit, repr) in actor_keywords {
        if lower.contains(kw) && !seen_role.contains_key(role) {
            let mut a = Actor::new(id, kw, role);
            a.participation_threshold = *pthr;
            a.exit_cost = *exit;
            a.voice = *voice;
            a.legitimacy = *legit;
            a.repression = *repr;
            // Capacity roughly tracks share.
            a.capacity = 1.0 + 2.0 * share;
            a.max_capacity = 2.0 * a.capacity;
            c.add_actor(a);
            seen_role.insert(role, id);
            id += 1;
        }
    }

    // Relation keyword -> (from_role, to_role, kind, output, share_from,
    //                      share_to, contribution_from, contribution_to).
    let relation_keywords: &[(&str, RelationKind, &str, &str, f64, f64, f64, f64)] = &[
        // -------- classical relations --------
```

```
        ("tribute", RelationKind::Tribute, "ruler", "producer", 1.0, 0.90, 0.10, 0.05, 0.08),
        ("tax", RelationKind::Taxation, "ruler", "producer", 1.0, 0.85, 0.15, 0.04, 0.12),
        ("corvee", RelationKind::Corvee, "ruler", "producer", 1.0, 0.90, 0.10, 0.02, 0.09),
        ("corvee", RelationKind::Corvee, "ruler", "producer", 1.0, 0.90, 0.10, 0.02, 0.09),
        ("slave", RelationKind::Slavery, "elite", "unfree", 1.0, 0.95, 0.05, 0.01, 0.15),
        ("rent", RelationKind::Rent, "elite", "producer", 1.0, 0.70, 0.30, 0.05, 0.28),
        ("feudal", RelationKind::FeudalService, "elite", "producer", 1.0, 0.70, 0.30, 0.05, 0.28),
        ("trade", RelationKind::Exchange, "trader", "trader", 1.0, 0.50, 0.50, 0.40, 0.40),
        ("liturgy", RelationKind::Liturgy, "elite", "member", 1.0, 0.30, 0.70, 0.15, 0.20),
        // -------- contemporary relations --------
        ("platform", RelationKind::PlatformLabor, "platform", "producer", 1.0, 0.90, 0.10, 0.05, 0.09),
        ("gig", RelationKind::PlatformLabor, "platform", "producer", 1.0, 0.90, 0.10, 0.05, 0.09),
        ("sovereign debt", RelationKind::SovereignDebt, "capital", "state", 1.0, 0.85, 0.15, 0.05, 0.14),
        ("austerity", RelationKind::SovereignDebt, "capital", "state", 1.0, 0.85, 0.15, 0.05, 0.14),
        ("supply chain", RelationKind::SupplyChainExtraction, "firm", "producer", 1.0, 0.85, 0.15, 0.05,
    ↪ 0.14),
        ("data", RelationKind::DataExtraction, "platform", "data subject", 1.0, 0.95, 0.05, 0.02, 0.045),
        ("carbon", RelationKind::CarbonExtraction, "firm", "biosphere", 1.0, 0.98, 0.02, 0.00, 0.06),
        ("tax haven", RelationKind::TaxHavenArbitrage, "capital", "state", 1.0, 0.90, 0.10, 0.05, 0.09),
        ("visa", RelationKind::VisaRegime, "state", "producer", 1.0, 0.80, 0.20, 0.05, 0.18),
        ("care", RelationKind::CareLabor, "household", "carer", 1.0, 0.90, 0.10, 0.05, 0.09),
        ("conflict mineral", RelationKind::ConflictMinerals, "firm", "producer", 1.0, 0.95, 0.05, 0.00,
    ↪ 0.06),
        ("fisheries", RelationKind::FisheriesDepletion, "firm", "biosphere", 1.0, 0.98, 0.02, 0.00, 0.06),
    ];

    // Find an actor by role, falling back to the ruler when the nominal
    // principal role is absent (e.g. slavery described without an explicit
    // elite class).
    let find = |c: &Civilization, role: &str| -> Option<ActorId> {
        c.actors
            .values()
            .find(|a| a.role == role)
            .or_else(|| c.actors.values().find(|a| a.role == "ruler"))
            .map(|a| a.id)
    };

    let mut rid = 0;
    let mut seen_kind: Vec<RelationKind> = Vec::new();
    for (kw, kind, from_role, to_role, output, sf, st, cf, ct) in relation_keywords {
        if lower.contains(kw) && !seen_kind.contains(kind) {
            let from = find(&c, from_role);
            let to = find(&c, to_role);
            if let (Some(f), Some(t)) = (from, to) {
                let mut r = Relation::new(rid, f, t, *kind, *output, *sf, *st, *cf, *ct);
                r.description = format!("Detected keyword: {}", kw);
                c.add_relation(r);
                seen_kind.push(*kind);
                rid += 1;
            }
        }
    }

    // Achievement keyword -> (domain, labor-years).
    let achievement_keywords: &[(&str, &str, f64)] = &[
        ("pyramid", "architecture", 100_000.0),
        ("temple", "architecture", 20_000.0),
        ("wall", "architecture", 50_000.0),
        ("canal", "engineering", 30_000.0),
        ("aqueduct", "engineering", 25_000.0),
        ("road", "engineering", 15_000.0),
        ("palace", "architecture", 10_000.0),
        ("code", "law", 500.0),
        ("library", "knowledge", 2_000.0),
        ("university", "knowledge", 5_000.0),
        ("cathedral", "architecture", 40_000.0),
        ("mosque", "architecture", 25_000.0),
        ("statue", "art", 5_000.0),
        ("monument", "architecture", 10_000.0),
    ];

    for (kw, domain, labor) in achievement_keywords {
        if lower.contains(kw) {
```

```
            c.add_achievement(Achievement {
                name: kw.to_string(),
                domain: domain.to_string(),
                labor_years: *labor,
                surplus_cost: *labor * 0.5,
                credited: vec![],
                laborers_named: vec![],
            });
        }
    }

    c
}

// ============================================================================
// 8. REPORT
// ============================================================================

pub fn render_report(c: &Civilization) -> String {
    let mut out = String::new();

    let _ = writeln!(out, "================================================================");
    let _ = writeln!(out, "CIVILIZATION ANALYSIS: {}", c.name);
    let _ = writeln!(out, "Period: {}", c.period);
    let _ = writeln!(out, "================================================================");
    let _ = writeln!(out);

    // -- Actors ------------------------------------------------
    let _ = writeln!(out, "ACTORS ({}):", c.actors.len());
    let mut ids: Vec<ActorId> = c.actors.keys().copied().collect();
    ids.sort();
    for id in &ids {
        let a = &c.actors[id];
        let _ = writeln!(
            out,
            " [{}] {} ({}) K={:.2} L={:.2} C={:.2} exit={:.2} voice={:.2}",
            a.id, a.name, a.role, a.capacity, a.legitimacy, a.conflict, a.exit_cost, a.voice
        );
    }
    let _ = writeln!(out);

    // -- Status ------------------------------------------------
    let status = assess_status(c);
    let _ = writeln!(out, "STATUS:");
    let _ = writeln!(out, " {}", status.aggregate_justification);
    for r in &status.relations {
        let _ = writeln!(
            out,
            " [{}] {} -> {} ({}): {:?} | net_from={:.3}, net_to={:.3}",
            r.id,
            r.from,
            r.to,
            r.kind.label(),
            r.class,
            r.net_from,
            r.net_to
        );
        if let Some(p) = &r.exploited_party {
            let _ = writeln!(out, "     exploited: {} ({:?})", p, r.exploited_flow.unwrap());
        }
    }
    let _ = writeln!(out);

    // -- Extraction bounds -------------------------------------
    let _ = writeln!(out, "EXTRACTION BOUNDS:");
    for id in &ids {
        let a = &c.actors[id];
        let bounds = extraction_bounds(a, 0.0);
        let _ = writeln!(
            out,
            " {}: monopolist={:.2}, exit ceiling={:.2}, effective={:.2} ({})",
            bounds.actor,
            bounds.monopolist_rate,
```

```
            bounds.exit_ceiling,
            bounds.effective_rate,
            bounds.binding_constraint
        );
    }
    let _ = writeln!(out);

    // -- Hidden history ---------------------------------------------
    let hidden = detect_hidden_history(c);
    let _ = writeln!(out, "HIDDEN HISTORY CANDIDATES ({}):", hidden.len());
    if hidden.is_empty() {
        let _ = writeln!(out, " (none detected)");
    }
    for (i, h) in hidden.iter().enumerate() {
        let _ = writeln!(out, " {}. {} [confidence {:.2}]", i + 1, h.headline, h.confidence);
        let _ = writeln!(out, "     Reasoning: {}", h.reasoning);
        if !h.implicated_actors.is_empty() {
            let _ = writeln!(out, "     Implicated: {}", h.implicated_actors.join(", "));
        }
        for e in &h.evidence {
            let _ = writeln!(out, "     - {}", e);
        }
    }
    let _ = writeln!(out);

    // -- Dynamics projection ----------------------------------------
    let params = Params::default();
    let traj = project(c, &params, 50.0);
    let _ = writeln!(out, "FUTURE DIRECTION:");
    let _ = writeln!(out, " Baseline phase: {}", traj.phase);
    let _ = writeln!(
        out,
        " Final state: K={:.2}, L={:.2}, C={:.2}, G={:.2}",
        traj.final_state.total_capacity,
        traj.final_state.avg_legitimacy,
        traj.final_state.avg_conflict,
        traj.final_state.injustice_gap
    );
    let _ = writeln!(
        out,
        " Distance to injustice threshold: {:.3}",
        traj.threshold_distance
    );
    if let Some(f) = &traj.counterintuitive_finding {
        let _ = writeln!(out, " NON-OBVIOUS FINDING: {}", f);
    }
    let _ = writeln!(out);

    // -- Counterfactual stress tests --------------------------------
    let _ = writeln!(out, "COUNTERFACTUAL STRESS TESTS:");
    for (label, delta_exit) in [
        ("Halve exit costs for all actors", -0.5),
        ("Double exit costs for all actors", 0.5),
    ] {
        let mut c2 = c.clone();
        for a in c2.actors.values_mut() {
            a.exit_cost = (a.exit_cost + delta_exit).clamp(0.0, 1.0);
        }
        let t2 = project(&c2, &params, 50.0);
        let _ = writeln!(
            out,
            " {}: phase={}, final K={:.2}, C={:.2}",
            label, t2.phase, t2.final_state.total_capacity, t2.final_state.avg_conflict
        );
    }
    let _ = writeln!(out);

    out
}

// ======================================================================
// 9. CONTEMPORARY DIAGNOSTICS (extension)
// ======================================================================
```

```rust

/// Exit asymmetry index: the ratio of labour's exit cost to capital's.
///
/// A >> 1 means capital is mobile and labour is captive. The extraction
/// ceiling on capital approaches zero; on labour, it approaches its maximum.
/// This is the central structural fact of the contemporary ledger.
pub fn exit_asymmetry_index(c: &Civilization) -> f64 {
    c.labor_exit_cost() / c.capital_exit_cost()
}

/// Detects compliance-by-atomization: an order in which legitimacy is low but
/// conflict is also low, because surveillance and narrative fragmentation
/// prevent the translation of grievance into collective action.
///
/// Formal condition (Proposition 16.3): L < L*, C < C*, R > R*.
pub fn detect_compliance_by_atomization(c: &Civilization) -> Option<String> {
    let avg_l: f64 = c.actors.values().map(|a| a.legitimacy).sum::<f64>()
        / c.actors.len().max(1) as f64;
    let avg_c: f64 =
        c.actors.values().map(|a| a.conflict).sum::<f64>() / c.actors.len().max(1) as f64;
    let avg_r: f64 =
        c.actors.values().map(|a| a.repression).sum::<f64>() / c.actors.len().max(1) as f64;

    if avg_l < 0.4 && avg_c < 0.1 && avg_r > 0.5 {
        return Some(format!(
            "Compliance by atomization detected. Legitimacy L={:.2} is low \
             and conflict C={:.2} is low, but repression R={:.2} is high. \
             The order is stable but not legitimate. A small shock to the \
             surveillance architecture -- a communication technology that \
             defeats it, a fiscal crisis that ends welfare residue, or a \
             legitimacy event -- can move the system to open conflict in a \
             short interval. The stability is real but thin.",
            avg_l, avg_c, avg_r
        ));
    }
    None
}

/// Detects biospheric predation: an order in which the ultimate host is unique
/// and non-substitutable, so the conservation corollary of Proposition 8.1
/// fails and extraction is self-terminating.
pub fn detect_biospheric_predation(c: &Civilization) -> Option<String> {
    let has_biospheric = c.relations.iter().any(|r| r.kind.is_biospheric());
    if !has_biospheric {
        return None;
    }

    let elite_impatience: f64 = c
        .actors
        .values()
        .filter(|a| a.role.contains("elite") || a.role.contains("ruler") || a.role.contains("capital"))
        .map(|a| 1.0 - a.legitimacy) // proxy for discount rate
        .fold(0.0_f64, f64::max);

    if elite_impatience > 0.5 {
        return Some(format!(
            "Biospheric predation detected. The host is unique and \
             non-substitutable. The conservation corollary fails: the \
             optimal extraction rate under a finite horizon is \
             self-terminating, not self-limiting. Elite impatience is \
             {:.2}. The framework predicts cascading failure across all \
             relations once the host's capacity crosses its threshold.",
            elite_impatience
        ));
    }
    None
}

/// Full contemporary report. Extends render_report with the modern diagnostics.
pub fn render_contemporary_report(c: &Civilization) -> String {
    let mut out = render_report(c);

    let _ = writeln!(out, "================================================================");
```

```
    let _ = writeln!(out, "CONTEMPORARY DIAGNOSTICS");
    let _ = writeln!(out, "================================================================");

    let a = exit_asymmetry_index(c);
    let _ = writeln!(out, "Exit asymmetry index: {:.2}", a);
    if a > 5.0 {
        let _ = writeln!(
            out,
            " Interpretation: capital is far more mobile than labour. \
             The extraction ceiling on labour is near its maximum; on \
             capital, near its minimum. This is the central structural \
             fact of the ledger. Rebalancing requires equalizing exit \
             costs, not merely redistributing shares."
        );
    } else if a < 1.5 {
        let _ = writeln!(
            out,
            " Interpretation: exit costs are relatively symmetric. \
             Extraction is bounded by exit for both factors. The ledger \
             is closer to a bargaining equilibrium."
        );
    }
    let _ = writeln!(out);

    if let Some(s) = detect_compliance_by_atomization(c) {
        let _ = writeln!(out, "NON-OBVIOUS FINDING:");
        let _ = writeln!(out, " {}", s);
        let _ = writeln!(out);
    }

    if let Some(s) = detect_biospheric_predation(c) {
        let _ = writeln!(out, "NON-OBVIOUS FINDING:");
        let _ = writeln!(out, " {}", s);
        let _ = writeln!(out);
    }

    out
}

// ============================================================================
// 10. EXAMPLES
// ============================================================================

fn classical_example() {
    let description = "
    The kingdom of Ur-Nammu was ruled by a king and a priestly class.
    Peasants paid tribute and were subject to corvee labor for the
    construction of canals, temples, and a great wall. Merchants
    engaged in trade along the river. Slaves were captured in war
    and worked the fields. The king issued a code of laws.
    ";

    let c = heuristic_parse(description, "Ur-Nammu", "Bronze Age");
    println!("{}", render_report(&c));
}

fn contemporary_example() {
    let description = "
    A global order in which capital moves freely across borders,
    while workers face visa regimes and high relocation costs.
    Multinational firms extract resources through supply chains
    that span jurisdictions. Platform companies employ gig workers
    without employment contracts. Data is extracted from users
    as a condition of service. Sovereign debt is held by bondholders
    who demand austerity. Carbon extraction continues at scale,
    degrading the biosphere. The state expands surveillance.
    Households depend on unpaid care. Tax havens allow firms to
    book profits where they do no work.
    ";

    let c = heuristic_parse(description, "Contemporary Order", "21st century");
    println!("{}", render_contemporary_report(&c));
}
```

```rust

fn main() {
    classical_example();
    println!("\n\n");
    contemporary_example();
}
```

B Sample Run

```
============================================================
CIVILIZATION ANALYSIS: Ur-Nammu
Period: Bronze Age
============================================================

ACTORS (5):
 [0] king (ruler) K=2.50 L=0.85 C=0.00 exit=0.00 voice=0.80
 [1] priest (clergy) K=1.30 L=0.80 C=0.00 exit=0.20 voice=0.50
 [2] merchant (trader) K=1.30 L=0.50 C=0.00 exit=0.40 voice=0.40
 [3] peasant (producer) K=1.06 L=0.35 C=0.00 exit=0.70 voice=0.10
 [4] slave (unfree) K=1.00 L=0.20 C=0.00 exit=1.00 voice=0.00

STATUS:
 1 symbiotic, 2 parasitic, 1 predatory relations. Aggregate classification: Predatory (worst category present).
 [0] king -> peasant (tribute): Parasitic | net_from=0.850, net_to=0.020
     exploited: peasant (Parasitic)
 [1] king -> peasant (corvee): Parasitic | net_from=0.880, net_to=0.010
     exploited: peasant (Parasitic)
 [2] king -> slave (slavery): Predatory | net_from=0.940, net_to=-0.100
     exploited: slave (Predatory)
 [3] merchant -> merchant (exchange): Symbiotic | net_from=0.100, net_to=0.100

EXTRACTION BOUNDS:
 king: monopolist=0.50, exit ceiling=0.68, effective=0.50 (revenue maximization)
 priest: monopolist=0.50, exit ceiling=0.29, effective=0.29 (exit)
 merchant: monopolist=0.50, exit ceiling=0.05, effective=0.05 (exit)
 peasant: monopolist=0.50, exit ceiling=0.00, effective=0.00 (exit)
 slave: monopolist=0.50, exit ceiling=0.00, effective=0.00 (exit)

HIDDEN HISTORY CANDIDATES (5):
 1. Unrecorded labor mobilization [confidence 0.70]
     Reasoning: Achievements imply far more labor than the visible coercive channels account for. The gap suggests unrecorded corvee, slavery, or tribute.
     Implicated: slave, peasant
     - Achievements require ~100500 labor-years; coercive channels can supply ~2.
     - No laborers are named in the official record of achievements.
 2. Suppressed grievance under tribute [confidence 0.53]
     Reasoning: peasant receives 0.030 below its participation threshold in a coercive relation. Grievance of this magnitude is typically suppressed in the official record.
     Implicated: peasant
     - Net gain 0.020 vs. threshold 0.050
 3. Suppressed grievance under corvee [confidence 0.54]
     Reasoning: peasant receives 0.040 below its participation threshold in a coercive relation. Grievance of this magnitude is typically suppressed in the official record.
     Implicated: peasant
     - Net gain 0.010 vs. threshold 0.050
 4. Suppressed grievance under slavery [confidence 0.62]
     Reasoning: slave receives 0.120 below its participation threshold in a coercive relation. Grievance of this magnitude is typically suppressed in the official record.
     Implicated: slave
     - Net gain -0.100 vs. threshold 0.020
 5. Captive population with no record of dissent [confidence 0.65]
     Reasoning: High exit costs and low voice imply that dissatisfaction cannot be expressed or observed. The absence of recorded grievance is evidence of suppression, not contentment.
     Implicated: slave, peasant
     - slave: exit cost 1.00, voice 0.00
     - peasant: exit cost 0.70, voice 0.10

FUTURE DIRECTION:
 Baseline phase: open conflict
```

```
 Final state: K=13.57, L=0.00, C=13.91, G=0.19
 Distance to injustice threshold: 0.010

COUNTERFACTUAL STRESS TESTS:
 Halve exit costs for all actors: phase=open conflict, final K=13.57, C=13.63
 Double exit costs for all actors: phase=open conflict, final K=13.31, C=14.21




============================================================
CIVILIZATION ANALYSIS: Contemporary Order
Period: 21st century
============================================================

ACTORS (8):
 [0] capital (capital) K=1.70 L=0.30 C=0.00 exit=0.02 voice=0.85
 [1] multinational (firm) K=1.60 L=0.35 C=0.00 exit=0.05 voice=0.80
 [2] platform (platform) K=1.60 L=0.40 C=0.00 exit=0.15 voice=0.60
 [3] state (state) K=1.50 L=0.40 C=0.00 exit=0.05 voice=0.70
 [4] worker (producer) K=1.08 L=0.30 C=0.00 exit=0.70 voice=0.15
 [5] user (data subject) K=1.04 L=0.30 C=0.00 exit=0.85 voice=0.05
 [6] household (household) K=1.10 L=0.40 C=0.00 exit=0.60 voice=0.20
 [7] biosphere (biosphere) K=1.00 L=0.20 C=0.00 exit=1.00 voice=0.00

STATUS:
 0 symbiotic, 6 parasitic, 1 predatory relations. Aggregate classification: Predatory (worst category
     ↪ present).
 [0] platform -> worker (platform labor): Parasitic | net_from=0.850, net_to=0.010
     exploited: worker (Parasitic)
 [1] capital -> state (sovereign debt): Parasitic | net_from=0.800, net_to=0.010
     exploited: state (Parasitic)
 [2] multinational -> worker (supply-chain extraction): Parasitic | net_from=0.800, net_to=0.010
     exploited: worker (Parasitic)
 [3] platform -> user (data extraction): Parasitic | net_from=0.930, net_to=0.005
     exploited: user (Parasitic)
 [4] multinational -> biosphere (carbon extraction): Predatory | net_from=0.980, net_to=-0.040
     exploited: biosphere (Predatory)
 [5] capital -> state (tax-haven arbitrage): Parasitic | net_from=0.850, net_to=0.010
     exploited: state (Parasitic)
 [6] state -> worker (visa regime): Parasitic | net_from=0.750, net_to=0.020
     exploited: worker (Parasitic)

EXTRACTION BOUNDS:
 capital: monopolist=0.50, exit ceiling=0.63, effective=0.50 (revenue maximization)
 multinational: monopolist=0.50, exit ceiling=0.55, effective=0.50 (revenue maximization)
 platform: monopolist=0.50, exit ceiling=0.37, effective=0.37 (exit)
 state: monopolist=0.50, exit ceiling=0.55, effective=0.50 (revenue maximization)
 worker: monopolist=0.50, exit ceiling=0.00, effective=0.00 (exit)
 user: monopolist=0.50, exit ceiling=0.00, effective=0.00 (exit)
 household: monopolist=0.50, exit ceiling=0.00, effective=0.00 (exit)
 biosphere: monopolist=0.50, exit ceiling=0.00, effective=0.00 (exit)

HIDDEN HISTORY CANDIDATES (8):
 1. Suppressed grievance under platform labor [confidence 0.54]
     Reasoning: worker receives 0.040 below its participation threshold in a coercive relation. Grievance
     ↪ of this magnitude is typically suppressed in the official record.
     Implicated: worker
     - Net gain 0.010 vs. threshold 0.050
 2. Suppressed grievance under sovereign debt [confidence 0.54]
     Reasoning: state receives 0.040 below its participation threshold in a coercive relation. Grievance
     ↪ of this magnitude is typically suppressed in the official record.
     Implicated: state
     - Net gain 0.010 vs. threshold 0.050
 3. Suppressed grievance under supply-chain extraction [confidence 0.54]
     Reasoning: worker receives 0.040 below its participation threshold in a coercive relation. Grievance
     ↪ of this magnitude is typically suppressed in the official record.
     Implicated: worker
     - Net gain 0.010 vs. threshold 0.050
 4. Suppressed grievance under data extraction [confidence 0.55]
     Reasoning: user receives 0.045 below its participation threshold in a coercive relation. Grievance of
     ↪  this magnitude is typically suppressed in the official record.
```

```
     Implicated: user
     - Net gain 0.005 vs. threshold 0.050
 5. Suppressed grievance under carbon extraction [confidence 0.54]
     Reasoning: biosphere receives 0.040 below its participation threshold in a coercive relation.
     ↪ Grievance of this magnitude is typically suppressed in the official record.
     Implicated: biosphere
     - Net gain -0.040 vs. threshold 0.000
 6. Suppressed grievance under tax-haven arbitrage [confidence 0.54]
     Reasoning: state receives 0.040 below its participation threshold in a coercive relation. Grievance
     ↪ of this magnitude is typically suppressed in the official record.
     Implicated: state
     - Net gain 0.010 vs. threshold 0.050
 7. Suppressed grievance under visa regime [confidence 0.53]
     Reasoning: worker receives 0.030 below its participation threshold in a coercive relation. Grievance
     ↪ of this magnitude is typically suppressed in the official record.
     Implicated: worker
     - Net gain 0.020 vs. threshold 0.050
 8. Captive population with no record of dissent [confidence 0.65]
     Reasoning: High exit costs and low voice imply that dissatisfaction cannot be expressed or observed.
     ↪ The absence of recorded grievance is evidence of suppression, not contentment.
     Implicated: worker, user, biosphere
     - worker: exit cost 0.70, voice 0.15
     - user: exit cost 0.85, voice 0.05
     - biosphere: exit cost 1.00, voice 0.00

FUTURE DIRECTION:
 Baseline phase: open conflict
 Final state: K=19.33, L=0.00, C=14.21, G=0.14
 Distance to injustice threshold: 0.055

COUNTERFACTUAL STRESS TESTS:
 Halve exit costs for all actors: phase=open conflict, final K=19.52, C=14.00
 Double exit costs for all actors: phase=open conflict, final K=18.45, C=14.52

============================================================
CONTEMPORARY DIAGNOSTICS
============================================================
Exit asymmetry index: 42.50
 Interpretation: capital is far more mobile than labour. The extraction ceiling on labour is near its
     ↪ maximum; on capital, near its minimum. This is the central structural fact of the ledger.
     ↪ Rebalancing requires equalizing exit costs, not merely redistributing shares.

NON-OBVIOUS FINDING:
 Compliance by atomization detected. Legitimacy L=0.33 is low and conflict C=0.00 is low, but repression R
     ↪ =0.54 is high. The order is stable but not legitimate. A small shock to the surveillance
     ↪ architecture -- a communication technology that defeats it, a fiscal crisis that ends welfare
     ↪ residue, or a legitimacy event -- can move the system to open conflict in a short interval. The
     ↪ stability is real but thin.

NON-OBVIOUS FINDING:
 Biospheric predation detected. The host is unique and non-substitutable. The conservation corollary fails
     ↪ : the optimal extraction rate under a finite horizon is self-terminating, not self-limiting. Elite
     ↪  impatience is 0.70. The framework predicts cascading failure across all relations once the host's
     ↪  capacity crosses its threshold.
```